



Universidad Nacional Autónoma de México
Escuela Nacional de Estudios Superiores
Unidad Morelia



Reporte de Proyecto 2
Encontrando Estructuras

PRESENTA:

Aguilar Uribe Alexis Uriel

NÚMERO DE CUENTA:

424060075

PROFESOR:

Dr. Tinoco Martínez Sergio Rogelio

Licenciatura en Tecnologías para la Información en Ciencias
Procesamiento Digital de Imágenes

A 12 de Octubre de 2026

Índice

1. Introducción	2
2. Planteamiento del Problema	3
2.1. Restricciones	3
3. Realización General del Proyecto	4
4. Resultados	5
4.1. Imagen 1	5
4.2. Imagen 2	6
4.3. Imagen 3	7
4.4. Imagen 4	9
4.5. Imagen 5	10
5. Conclusiones	12
6. Declaración de Uso Responsable de IA	13
Referencias	14

1. Introducción

Entre las operaciones de preprocesamiento para los problemas de localización y segmentación se encuentra la detección de bordes, esta técnica permite aislar de forma preliminar los objetos que están presentes en una imagen. Por lo tanto, el problema de los bordes no solamente se limita a una curiosidad en visión computacional, sino que se vuelve un proceso intermedio para mejorar y optimizar los modelos de IA más sofisticados como las CNN (Convolutional Neural Network). Esta se vuelve la principal motivación para entender cómo surgen los bordes, cómo se pueden detectar y, aún más importante, cómo se pueden integrar estas ideas en redes neuronales.

Para realizar este proyecto, se hace uso de los filtros de Prewitt, Sobel, Laplace, Laplaciano de Gaussiana y Canny para compararlos en diferentes escenarios de bordes e imágenes con y sin suavizado gaussiano con el fin de entender sus ventajas, desventajas y cómo los bordes son afectados (su definición y ruido presente) por el filtro específico que se emplea.

De forma general, los filtros de Prewitt y Sobel permiten detectar bordes geométricos sin dificultad, mientras que los filtros de Laplace y Laplaciano de Gaussiana les va mejor en bordes curvos o no geométricos (Tinoco, 2026). El filtro de Canny emplea de los filtros de Prewitt o de Sobel para detectar bordes horizontales, verticales y diagonales, esto favorece a que este filtro sea el más conveniente para todos los escenarios, en teoría (Wikipedia contributors, 2026).

2. Planteamiento del Problema

Para un conjunto de imágenes se deberá comparar:

1. Métodos de detección
2. Ventajas y desventajas de los diferentes métodos

Así como comparar el resultado de aplicar los siguientes tipos de suavizados sobre la imagen (para cada método de detección), analizando el efecto sobre los bordes:

- Sin suavizado.
- Con suavizado gaussiano con un kernel de 3×3 píxeles.
- Con suavizado gaussiano con un kernel de 7×7 píxeles.

Adicionalmente, se deberá incluir la interpretación de las siguientes expresiones:

- $G_x = \frac{\partial I(x,y)}{\partial x}$
- $G_y = \frac{\partial I(x,y)}{\partial y}$
- $G = \sqrt{G_x^2 + G_y^2}$
- $\theta = \arctan\left(\frac{G_y}{G_x}\right)$

Por último, se visualizará la orientación de los bordes colocando una flecha con inicio en el pixel actual, de longitud conveniente, que apunte en la dirección señalada por el ángulo calculado θ .

2.1. Restricciones

Al menos Sobel y Prewitt deberán implementarse manualmente con NumPy, sin utilizar una función de alto nivel que haga el procedimiento.

3. Realización General del Proyecto

Primero se presentan las interpretaciones de los términos relacionados a la detección de bordes. Estas interpretaciones se basan en conocimientos previos y en lo visto en clases (Tinoco, 2026):

- $G_x = \frac{\partial I(x,y)}{\partial x}$: Indica cómo cambia la intensidad de los colores de forma horizontal. Estos cambios permiten encontrar líneas verticales.
- $G_y = \frac{\partial I(x,y)}{\partial y}$: Indica cómo cambia la intensidad de los colores de forma vertical. Estos cambios permiten encontrar líneas horizontales.
- $G = \sqrt{G_x^2 + G_y^2}$: Representan la magnitud del cambio de la intensidad de los colores de forma direccional. Es la magnitud de la derivada direccional.
- $\theta = \arctan(\frac{G_y}{G_x})$: Es la dirección hacia donde la intensidad de los colores cambia “más fuerte”. Es el ángulo de la derivada direccional.

La implementaciones de las funciones relacionadas a los filtros se basan en las de CV2 (Bradski, 2000), es decir, se verificó que las funciones programadas con NumPy (Harris et al., 2020) coincidan en valores con las de CV2. Aunque estas funciones implementadas en NumPy nunca son llamadas, debido a que son “lentas” en comparación con su equivalente en CV2 que se ejecuta en Cpp. Por ello, solamente se manejan estas funciones con los parámetros adecuados para generar los resultados esperados que son requeridos para la práctica.

Todas las matrices asociadas a los filtros revisados en clase fueron implementados de forma manual, salvo los asociados a los suavizados gaussianos. Estos filtros fueron comparados sobre las diferentes variantes de suavizados aplicadas a las imágenes que, de forma general, permitieron reducir el ruido en regiones con texturas de alto contraste que podrían confundirse con bordes, es decir, píxeles aislados que son tratados como bordes. En algunos casos, el suavizado favoreció a encontrar bordes más definidos, en otros simplemente los difuminaron.

4. Resultados

En la presente sección se exponen los resultados más relevantes de cada imagen del conjunto, destacando los filtros que obtuvieron mejores resultados, así como explicando sus ventajas y desventajas específicas en cada imagen o escenario.

4.1. Imagen 1

Esta imagen contiene elementos tanto geométricos como no geométricos con texturas muy variables. Esto significa que los filtros que sean sensibles al ruido, van a detectar muchos no bordes, como le pasa principalmente al filtro de Laplaciano de Gaussiana; esto también ocurre en los filtros de Prewitt y Sobel. Pero este fenómeno se puede mitigar con un adecuado suavizado gaussiano.

Imagen Original y a Escala de Grises de la Imagen 1



Figura 1: Imagen 1 del conjunto.

Comparación del Filtro Laplaciano de Gaussiana de la Imagen 1

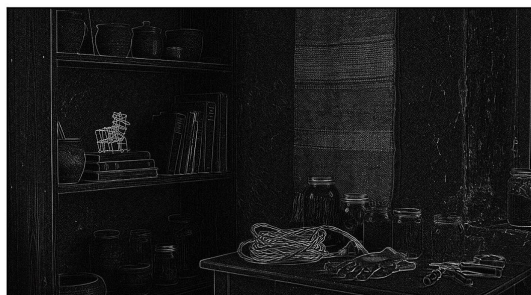


Figura 2: Filtro Laplaciano de Gaussiana sobre la Imagen 1 sin suavizado.

Al aplicar un suavizado gaussiano con un kernel de 7×7 , se pueden mejorar los bordes detectados, haciendo que se vean más definidos o sin tanto ruido debido a las texturas. Donde más se muestra una mejora es en el filtro de Canny, debido a que detecta adecuadamente los bordes sin

añadir ruido por la textura de los elementos, dicho así, el combinar la información de diferentes filtros le favorece para incrementar la sensibilidad al momento de detectar un verdadero borde.

Comparación del Filtro Canny de la Imagen 1 suavizado 7x7



Figura 3: Filtro de Canny sobre la Imagen 1 suavizada con un kernel 7×7 .

4.2. Imagen 2

Esta imagen cuenta con dos peculiaridades: tiene una sobreexposición y vegetación que añade mucha texturas y contraste de colores (áreas oscuras y claras). Por lo tanto, para reducir el efecto del brillo, se le aplicó una función de ganancia-sesgo, esto va a favorecer a encontrar algunos de los bordes empleando los filtros adecuados. Sin aplicar un suavizado gaussiano, cualquier filtro detecta parte del ruido presente, donde el filtro Laplaciano de Gaussiana es el más sensible a este efecto, y los filtros de Prewitt, Sobel y Canny manejan de mejor manera el ruido.

Imagen Original y a Escala de Grises de la Imagen 2 enhancement



Figura 4: Imagen 2 del conjunto.

Comparación del Filtro Prewitt de la Imagen 2



Figura 5: Filtro Prewitt sobre la Imagen 2 sin suavizado.

Después de aplicar un suavizado gaussiano con un kernel de 7×7 , es posible reducir el efecto de la textura de la vegetación y se aprovecha de mejor manera los filtros de Sobel y Prewitt para delimitar los elementos. Con este suavizado, los filtros de Laplace y Laplaciano de Gaussiana ignoran los cambios más sensibles de los colores (o locales).

Comparación del Filtro Sobel de la Imagen 2 suavizado 7×7

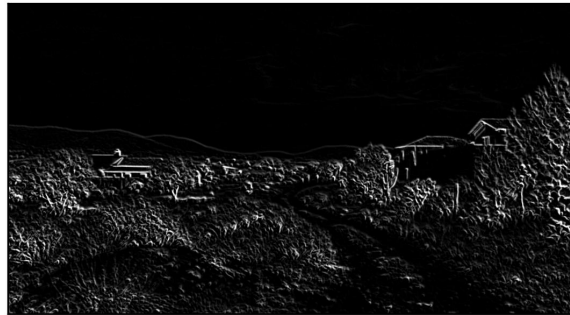


Figura 6: Filtro de Sobel sobre la Imagen 2 suavizada con un kernel 7×7 .

4.3. Imagen 3

Esta imagen dominan los elementos geométricos, esto permite que los filtros de Prewitt, Sobel y Canny detecten los bordes adecuadamente, mientras que los otros filtros puede que detecten algo de ruido por las texturas de las maderas. Con el fin de facilitar la detección de bordes, se mejora el brillo de la imagen con la función de ganancia-sesgo. Sin aplicar un suavizado, los filtros mencionados detectan los bordes adecuadamente, pero capturan algo de ruido por la textura de los elementos, siendo esta su mayor desventaja al ser sensibles a los cambios entre

píxeles. Mientras que, en este caso, el filtro de Canny se vuelve el más robusto contra el ruido y funciona adecuadamente para ignorar falsos bordes.

Imagen Original y a Escala de Grises de la Imagen 3 enhancement



Figura 7: Imagen 3 del conjunto.

Comparación del Filtro Canny de la Imagen 3



Figura 8: Filtro Canny sobre la Imagen 3 sin suavizado.

Con un suavizado gaussiano con kernel de 3×3 , se logra que el filtro Laplaciano de Gaussiana ignore ligeramente la textura de los elementos y detecte adecuadamente los bordes, notando así que este filtro es sensible al ruido o cambios locales de colores.

Comparación del Filtro Laplaciano de Gaussiana de la Imagen 3 suavizado 3×3



Figura 9: Filtro de Laplaciano de Gaussiana sobre la Imagen 3 suavizada con un kernel 3×3 .

4.4. Imagen 4

Esta imagen está dominada por elementos geométricos (rectas), esto permite que todos los filtros detecten los bordes más relevantes que constituyen a la imagen. Aunque cuenta con zonas oscuras y claras, éstas no afectan a la detección de bordes, debido a que no obstruyen a las diferencias entre tonalidades (entre un ventana y una pared). En el caso de los filtros basados en la segunda derivada, tienen la desventaja de captura algo del ruido de las texturas, mientras que Prewitt y Sobel delimitan adecuadamente los bordes de los elementos.

Imagen Original y a Escala de Grises de la Imagen 4



Figura 10: Imagen 4 del conjunto.

Comparación del Filtro Laplaciano de Gaussiana de la Imagen 4



Figura 11: Filtro Laplaciano de Gaussiana sobre la Imagen 4 sin suavizado.

Reduciendo el ruido con un suavizado gaussiano con kernel de 7×7 , se logra que el filtro de Canny detecte los bordes de los elementos principales que se aprecian con un buen brillo, es decir, aquellos que están en la zonas claras sin que las texturas creen falsos bordes.

Comparación del Filtro Canny de la Imagen 4 suavizado 7x7



Figura 12: Filtro de Canny sobre la Imagen 4 suavizada con un kernel 7×7 .

4.5. Imagen 5

Aunque en esta imagen los elementos principales tiene una buena diferenciación de tonalidades, podemos notar que también cuentan con muchas texturas que harán que cualquier filtro las vea como bordes. Por ello, sin aplicar el suavizado cualquier filtro detecta esos cambios menores entre colores, lo que añade ruido a los bordes de forma innecesaria.

Imagen Original y a Escala de Grises de la Imagen 7



Figura 13: Imagen 5 del conjunto.

Comparación del Filtro Prewitt de la Imagen 7



Figura 14: Filtro Prewitt sobre la Imagen 5 sin suavizado.

Aplicando un suavizado gaussiano con kernel de 7×7 , permite detectar bordes correctamente sin el ruido añadido por las texturas de los diferentes elementos presentes en la imagen. En este caso, los filtros de Prewitt, Sobel y Canny detectan los bordes relevantes de los objetos sin dificultades, mientras que los filtros de Laplace y Laplaciano de Gaussiana no encuentran todos los bordes, tal vez debido a que el suavizado difuminó demasiado las tonalidades.

Comparación del Filtro Canny de la Imagen 7 suavizado 7x7

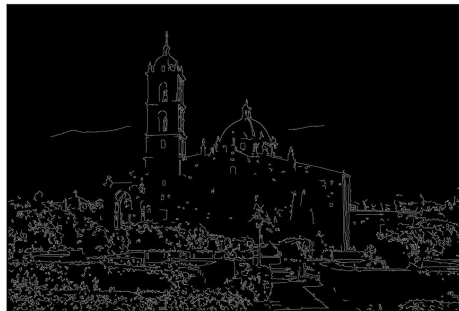


Figura 15: Filtro de Canny sobre la Imagen 5 suavizada con un kernel 7×7 .

5. Conclusiones

De las imágenes consideradas para el proyecto, me permitieron darme cuenta de que en la vida real está llena de muchas y diferentes texturas que para estos filtros pueden verse como si fueran bordes, por ello se vuelve vital aplicar una técnica de suavizado para reducir el efecto o implicaciones de considerar estos cambios menores en las tonalidades. Por ello, después del suavizado, los algoritmos como Prewitt, Sobel y Canny generan mejores bordes, debido a que la diferencia entre colores cercanos se redujo o es menor. El problema de los filtros de Prewitt y Sobel es que no son sensibles a las curvas, es decir, se les dificulta detectar elementos no rectos, pero esta es la principal ventaja que tienen los filtros de Laplace y Laplaciano de Gaussiana, son muy sensibles a los cambios de tonalidades (pues comparan vecinos adyacentes de píxeles) por ello pueden detectar curvas muy cerradas con una única pasada, es decir, no se requiere de dos “versiones” del mismo filtro sino con una única versión se puede detectar cualquier curva. Por esta misma sensibilidad, es que cualquier cambio en la textura o ruido se verá reflejado inmediatamente como un borde, para sanar esto podríamos introducir un umbral para determinar qué sí es un borde, y esta es la idea de filtro de Canny que emplea de varios filtros direccionales para detectar la mayoría de los bordes en una imagen (sin importar si son rectos o curvos). Por cómo se define, el filtro de Canny es el más complejo de usar y, personalmente, en el proyecto no pude usarlo debidamente porque no pude ajustar los umbrales adecuadamente sin que se descartara un borde o se introducieran no bordes (ruido).

Este miniproyecto se me hizo interesante el como existen métodos no tan sofisticados que pueden ser usados en etapas de preprocesamiento para mejorar la detección, localización y segmentación de objetos en tareas de visión por computadora empleando redes neuronales. Dicho así, no todo se trata de un modelo de IA que lo haga todo, sino también hay que entender y analizar el problema desde otras perspectivas o métodos que puedan integrarse a estos modelos. Y esto es lo que más me llevo de este proyecto, de realizar un paso hacia atrás para comprender un problema desde otra perspectiva empleando técnicas, métodos o modelos más sencillos/intuitivos y explicables (que no sean una caja negra) y que como científicos de datos podamos sacarle toda el conocimiento posible para seguir mejorando la solución final al problema que queremos resolver, evitando dejar huecos teóricos o sin considerar.

6. Declaración de Uso Responsable de IA

Se empleó de IA únicamente para comprender de mejor manera cómo está implementado el algoritmo de Canny en CV2, así como para entender qué argumentos admiten algunas de las funciones de CV2 que se emplearon para obtener las derivadas direccionales de las diferentes imágenes. Específicamente, se usó el modelo Gemini 3.8 Flash de Google (Google DeepMind, 2026).

Referencias

- Tinoco, S. (2026). Material de Detección de Bordes. <https://classroom.google.com/c/ODc0NzY1MDk1NzQ4>
- Wikipedia contributors. (2026). Canny edge detector. https://en.wikipedia.org/wiki/Canny_edge_detector
- Bradski, G. (2000). The OpenCV Library. *Dr. Dobb's Journal of Software Tools*.
- Harris, C. R., Millman, K. J., van der Walt, S. J., Gommers, R., Virtanen, P., Cournapeau, D., Wieser, E., Taylor, J., Berg, S., Smith, N. J., Kern, R., Picus, M., Hoyer, S., van Kerkwijk, M. H., Brett, M., Haldane, A., Fernández del Río, J., Wiebe, M., Peterson, P., ... Oliphant, T. E. (2020). Array programming with NumPy. *Nature*, 585, 357-362. <https://doi.org/10.1038/s41586-020-2649-2>
- Google DeepMind. (2026). *Gemini 3.8 Flash* (Ver. 3.8 Flash) [Large language model; Accessed: 2026-09-16]. Google. <https://deepmind.google>